

TurtleBot3 Burger

SLAM · AMCL · Navigation2 튜토리얼

cchyun@gmail.com

Part 1. 환경 구축

- 공식 레퍼런스 문서: [TurtleBot3 e-Manual 공식 가이드 PC Setup](#)
- 필수 소스 코드 저장소 (ROS2 Humble 브랜치)
 - `DynamixelSDK`: 로봇 모터 제어를 위한 통신 라이브러리
 - `turtlebot3_msgs`: TurtleBot3 전용 통신 메시지 패키지
 - `turtlebot3`: 로봇 구동, 3D 모델링, 키보드 조종, SLAM 및 내비게이션 등 핵심 기능을 포함하는 패키지
 - `turtlebot3_simulations`: Gazebo 시뮬레이션을 위한 가상 환경(월드 맵) 리소스

TurtleBot3 패키지 소스 빌드

워크스페이스 디렉토리 생성 및 소스 다운로드

```
mkdir -p ~/Workspace/turtlebot3_ws/src  
cd ~/Workspace/turtlebot3_ws/src/
```

```
git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git  
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git  
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git  
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
```

의존성 빌드 수행

```
cd ~/Workspace/turtlebot3_ws  
colcon build --symlink-install
```

- Gazebo 시뮬레이터 연동 설정

```
echo 'source /usr/share/gazebo/setup.sh' >> ~/.bashrc
```

- 개발 워크스페이스 로드 및 로봇 플랫폼 모델 설정

```
echo 'source ~/Workspace/turtlebot3_ws/install/setup.bash' >> ~/.bashrc  
echo 'export TURTLEBOT3_MODEL=burger' >> ~/.bashrc  
source ~/.bashrc
```

워크스페이스 주요 패키지 (1)

- `DynamixelSDK` (모터 제어 통신 라이브러리)

패키지명	설명
<code>dynamixel_sdk</code>	ROS 2용 다이나믹셀 제어 API 제공
<code>dynamixel_sdk_custom_interfaces</code>	사용자 정의 인터페이스 예제
<code>dynamixel_sdk_examples</code>	다이나믹셀 제어 동작 예제 코드

- `turtlebot3_msgs` (사용자 정의 메시지)

패키지명	설명
<code>turtlebot3_msgs</code>	TurtleBot3 전용 커스텀 메시지 및 서비스 타입 정의

워크스페이스 주요 패키지 (2)

- `turtlebot3` (핵심 구동 패키지)

패키지명	설명
<code>turtlebot3_bringup</code>	로봇 센서 및 모터 구동 런치 파일 (<code>robot.launch.py</code> 등)
<code>turtlebot3_cartographer</code>	Cartographer SLAM 매핑 실행 스크립트
<code>turtlebot3_description</code>	시각화를 위한 3D 모델(URDF) 정보
<code>turtlebot3_example</code>	상호작용 마커, 위치 제어 등 기초 예제 코드
<code>turtlebot3_navigation2</code>	Navigation2 자율주행 실행 스크립트
<code>turtlebot3_node</code>	양륜 구동 제어 및 오도메트리 연산 메인 노드
<code>turtlebot3_teleop</code>	키보드 원격 조종 제어 노드

워크스페이스 주요 패키지 (3)

- `turtlebot3_simulations` (가상 환경 시뮬레이션)

패키지명	주요 역할 및 런치 파일
<code>turtlebot3_fake_node</code>	물리 시뮬레이션 없이 동작을 테스트하는 가상 노드
<code>turtlebot3_gazebo</code>	Gazebo 가상 환경 실행 (<code>turtlebot3_world</code> , <code>empty_world</code> 등)
<code>turtlebot3_manipulation_gazebo</code>	매니퓰레이터(로봇 팔)가 결합된 가상 환경

Part 2. SLAM — 지도 만들기

- 터미널 1 — Gazebo 가상 환경 실행

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

- Gazebo 화면이 열리고 기동 장애물들이 있는 가상 실습 환경이 나타남

- 터미널 2 — Cartographer SLAM 실행

```
sudo apt install -y ros-humble-cartographer ros-humble-cartographer-ros  
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

- 주의: 시뮬레이터 환경에서 실행하므로 `use_sim_time:=True` 옵션을 반드시 지정해야 함

• Google Cartographer란?

- Google이 개발한 오픈소스 실시간 SLAM 라이브러리로, 2D 및 3D 환경 모두 지원
- 내부적으로 서브맵(Submap) 기반 매핑 + 글로벌 포즈 그래프 최적화의 2단계 구조를 채택
- 출력 (`.pbstream`): 서브맵 + 포즈 그래프 + 원시 스캔 데이터가 모두 포함된 직렬화 파일
- `cartographer_pbstream_to_ros_map` 도구로 `.pgm` + `.yaml` Nav2 호환 맵으로 변환 가능

• 핵심 동작 원리

단계	처리 내용	세부 설명
Local SLAM	서브맵 생성	최신 LiDAR 스캔을 Ceres Solver 기반 스캔 매칭으로 현재 서브맵에 삽입
Global SLAM	포즈 그래프 최적화	완성된 서브맵들 간의 제약 조건(Constraint)을 수집하고, 전역 그래프 최적화로 누적 드리프트를 일괄 보정
Loop Closure	루프 감지 및 보정	이전에 방문했던 장소인지 파악하여 누적된 위치 오차를 수정

키보드 조작 매핑 수행 및 지도 파일 저장

- 터미널 3 - 키보드 조종 제어 노드 가동

```
ros2 run turtlebot3_teleop teleop_keyboard
```

- 터미널 4 - SLAM 지도 저장 프로세스 (Pose Graph -> PGM/YAML)

```
# SLAM 최적화 그래프 파일(.pbstream)로 1차 저장
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \
  "{filename: '/home/cchyun/Workspace/turtlebot3_ws/map/tb3_map.pbstream'}"

# pbstream 리소스를 ROS2 내비게이션 맵 포맷(PGM 이미지 및 YAML 메타데이터)으로 변환
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename /home/cchyun/Workspace/turtlebot3_ws/map/tb3_map.pbstream \
  -map_filestem /home/cchyun/Workspace/turtlebot3_ws/map/tb3_map \
  -resolution 0.05
```

- 간단한 명령어로 지도 저장하기 (Nav2 Map Saver CLI)

```
ros2 run nav2_map_server map_saver_cli -f ~/Workspace/turtlebot3_ws/map/tb3_map
```

- 변환 결과 파일의 구성 형태

출력 파일명	리소스 세부 역할
tb3_map.pgm	지도의 형태를 나타내는 이미지 파일 (검은색: 장애물, 흰색: 이동 가능 영역, 회색: 미탐색 영역)
tb3_map.yaml	지도 해상도, 기준 좌표 등 지도의 설정 정보가 담긴 파일
tb3_map.pbstream	Cartographer 포즈 그래프 보존 원본 파일 (추후 확장 매핑 시 이어서 병합 가능)

- 저장된 지도를 불러와 정상 동작하는지 확인

터미널 1 : Gazebo 가상 환경 구동

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

터미널 2 : 절대 좌표 연동용 고정 변환(Static TF) 브로드캐스터 임시 가동 (초기 좌표 적용)

```
ros2 run tf2_ros static_transform_publisher 2 0.5 0 0 0 0 map odom
```

터미널 3 : 맵 서버 노드 가동

```
ros2 run nav2_map_server map_server \
```

```
--ros-args -p yaml_filename:=/home/cchyun/Workspace/turtlebot3_ws/map/tb3_map.yaml -p use_sim_time:=True
```

터미널 4 : 라이프사이클 관리 노드로 가동 개시 트리거

```
ros2 run nav2_lifecycle_manager lifecycle_manager \
```

```
--ros-args -p node_names:=['map_server'] -p autostart:=True -p use_sim_time:=True
```

터미널 5 : rviz 시각화

```
rviz2
```

Part 3. Cartographer RViz 시각화 상세

- **개요:** 라이다 원본 데이터가 Cartographer의 서브맵(부분 지도)에 매칭되는 과정을 보여주는 실시간 포인트 클라우드
- **기본 정보 및 속성**

항목	설정값 및 상세 내용
토픽 (Topic)	<code>/scan_matched_points2</code> (<code>sensor_msgs/PointCloud2</code>)
RViz 플러그인	<code>rviz_default_plugins/PointCloud2</code>
시각화 속성	색상: 초록색 (RGB 0, 255, 0) / 포인트 크기: 0.01m (Boxes)

- **분석 팁:** 초록색 점들이 기존 지도의 벽면 및 장애물 테두리와 일치하면 스캔 매칭 정상 작동을 의미함. 지도 범위를 크게 벗어날 경우 오도메트리 또는 라이다 매칭 오차 발생을 의미함.

- **개요:** 로봇이 생성한 주행 궤적과 각 궤적 노드(Node)들을 그래프 형태로 시각화
- **기본 정보 및 속성**

항목	설정값 및 상세 내용
토픽 (Topic)	<code>/trajectory_node_list</code> (<code>visualization_msgs/MarkerArray</code>)
RViz 플러그인	<code>rviz_default_plugins/MarkerArray</code>
시각화 속성	Cartographer 노드 내부 로직에 의해 결정 (주로 점과 선을 이용한 궤적 마커)

- **분석 팁:** 디버깅 시 활성화하여 로봇이 내부적으로 생성한 노드 수 및 경로 최적화 과정을 직관적으로 파악 가능

Cartographer RViz 시각화: Constraints (제약 조건)

- **개요:** Cartographer 최적화의 핵심인 제약 조건(Constraints) 상태를 시각화함
- **기본 정보 및 속성**

항목	설정값 및 상세 내용
토픽 (Topic)	<code>/constraint_list</code> (<code>visualization_msgs/MarkerArray</code>)
RViz 플러그인	<code>rviz_default_plugins/MarkerArray</code>
시각화 속성	Cartographer 최적화 계산 메시지에 포함된 색상 정보 사용

- 주요 용어 정의

- **제약 조건 (Constraint):** "이 서브맵(부분 지도)에서 로봇 위치(노드)는 이쯤에 연결되어야 해"라고 예측한 **가상의 연결 고리**
- **잔차 (Residual):** 제약 조건 예측 위치와 최종 최적화(Optimization)를 거쳐 계산된 실제 위치 간의 **어긋남(오차)**. SLAM 최적화가 잘 진행될수록 잔차 선은 점차 짧아짐.
- **Intra (내부):** 하나의 동일한 서브맵 내부에서 맺어지는 관계 (로컬 최적화 영역)
- **Inter (외부):** 서로 다른 서브맵 사이, 혹은 서로 다른 로봇/주행 경로(궤적) 사이에서 맺어지는 글로벌 관계 (최적화 영역)

제약 조건 시각화: Intra (서브맵 내부) 관계

- Intra constraint (서브맵 내부 제약 조건)

- 시작 / 끝점: 서브맵 기준 원점 (`submap_pose`) → 제약 조건 예측 노드의 글로벌 위치
- 색상: 서브맵마다 고유하게 부여되는 **다양한 파스텔 색상** (동적 변경)
- 의미: 동일 서브맵 내에서 스캔 매칭을 통해 계산된 노드(로봇 Pose) 간의 상대적 연결 고리

- Intra residual (서브맵 내부 오차)

- 시작 / 끝점: 제약 조건 예측 노드의 글로벌 위치 → 최적화 후 최종 결정된 실제 노드 글로벌 위치
- 색상: **빨간색**
- 의미: 센서 노이즈나 오도메트리의 미세 누적 오차 등으로 인해 발생한 예측과 실제 위치의 차이(잔차)

제약 조건 시각화: Inter Same (동일 경로 내 서브맵 간) 관계

- **Inter same constraint (동일 경로 내 서브맵 간 제약 조건)**
 - 시작 / 끝점: 연결된 과거 서브맵 기준 원점 → 제약 조건 예측 노드의 글로벌 위치
 - 색상: 밝은 노란색
 - 의미: 동일 경로 내 다른 서브맵과 노드를 연결.
 - 인접 연결: 단순 주행 시 이전 서브맵과 현재 서브맵을 이어줌
 - 루프 클로저: 이전에 방문한 장소로 되돌아왔을 때 과거 서브맵과 연결하여 전역 최적화 유도
- **Inter same residual (동일 경로 내 서브맵 간 오차)**
 - 시작 / 끝점: 제약 조건 예측 노드의 글로벌 위치 → 최적화 후 최종 결정된 실제 노드 글로벌 위치
 - 색상: 청록색 (Cyan)
 - 의미: 예측값과 실제 노드 위치 간 오차. 최적화가 정상 진행되어 지도가 정렬될수록 선이 급격히 짧아짐

제약 조건 시각화: Inter Diff (서로 다른 경로 간) 관계

- Inter diff constraint (서로 다른 경로 간 제약 조건)

- 시작 / 끝점: 첫 번째 주행 경로 서브맵 기준 원점 → 제약 조건 예측 두 번째 주행 경로 노드 위치
- 색상: 주황색
- 의미: 다중 로봇 SLAM 또는 여러 세션으로 주행한 지도를 하나로 합칠(Merge) 때 궤적 간 맺어지는 제약 조건 (단일 로봇 단일 세션 SLAM에서는 나타나지 않음)

- Inter diff residual (서로 다른 경로 간 오차)

- 시작 / 끝점: 제약 조건 예측 상대 궤적 노드 위치 → 최적화 후 최종 실제 상대 궤적 노드 위치
- 색상: 청록색 (Cyan)
- 의미: 서로 다른 주행 경로 지도 병합 시 좌표계 정렬 및 병합의 오차(잔차). (단일 로봇 단일 세션 SLAM 미표시)

- **개요:** 외부 랜드마크(예: AR 태그, 비콘 등) 연동 SLAM 수행 시 랜드마크의 추정 위치를 표시함
- **기본 정보 및 속성**

항목	설정값 및 상세 내용
토픽 (Topic)	<code>/landmark_poses_list</code> (<code>visualization_msgs/MarkerArray</code>)
RViz 플러그인	<code>rviz_default_plugins/MarkerArray</code>
시각화 속성	랜드마크 좌표계를 나타내는 축(Axis) 또는 형태 마커로 표시

- **분석 팁:** 기본 TurtleBot3 라이다 SLAM 예제에서는 미사용으로 화면에 나타나지 않으나, 추후 비전 랜드마크 융합 SLAM 시 활용 가능

Part 4. AMCL — 저장된 지도 위에서 위치 추정

- AMCL (Adaptive Monte Carlo Localization) 알고리즘이란?
 - 저장된 지도를 바탕으로 현재 LiDAR 센서 데이터를 비교하여, 로봇의 현재 위치를 확률적으로 역추정하는 알고리즘 (입자 필터 방식 사용)

- 터미널 1 — Gazebo 가상 환경 구동

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

- 터미널 2 — 정적 맵 서버 노드 가동

```
ros2 run nav2_map_server map_server \  
  --ros-args -p yaml_filename:=/home/cchyun/Workspace/turtlebot3_ws/map/tb3_map.yaml -p use_sim_time:=True
```

- 터미널 3 — AMCL 위치 추정 노드 가동

```
source ~/Workspace/turtlebot3_ws/install/setup.bash
ros2 run nav2_amcl amcl --ros-args -p use_sim_time:=True \
  -p base_frame_id:=base_footprint -p odom_frame_id:=odom -p global_frame_id:=map \
  -p robot_model_type:=nav2_amcl::DifferentialMotionModel
```

- 터미널 4 — Lifecycle Manager 기동 (동적 노드 상태 활성화)

```
ros2 run nav2_lifecycle_manager lifecycle_manager \
  --ros-args -p node_names:=['map_server', 'amcl'] -p autostart:=True -p use_sim_time:=True
```

터미널 5 — RViz2 모니터링 셋업

```
rviz2
```

디스플레이 항목	구독 토픽 (Topic)	고유 세부 설정
Map	/map	Durability: Transient Local 설정 변경
LaserScan	/scan	기본값 유지
TF	—	기본값 유지
PoseArray	/particlecloud	AMCL 파티클 필터 가상 후보군 분포 가시화

로봇의 초기 위치 설정하기

- 1. RViz2 화면 상단의 2D Pose Estimate 버튼 클릭
- 2. 지도에서 실제 로봇 위치를 클릭하고, 마우스 드래그로 로봇이 바라보는 방향 설정
- 3. 초록색 화살표(파티클)들이 지정한 위치 주변으로 조밀하게 모이면 위치 설정 성공

- 터미널 6 — Teleop 주행

```
ros2 run turtlebot3_teleop teleop_keyboard
```

- 로봇을 천천히 이동시키며 초록색 화살표들이 로봇 주변으로 잘 따라오는지 확인

- 로봇 위치 강제 초기화 (Global Localization) 실습

- 로봇이 현재 위치를 완전히 잃어버린 상황(납치된 로봇 문제, Kidnapped Robot Problem) 가정

```
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty {}
```

- 명령어 실행 시 초록색 화살표가 지도 전체에 무작위로 고르게 퍼짐
- 로봇을 제자리에서 회전시키면 센서 데이터가 지도와 일치하는 곳을 찾아 화살표들이 실제 위치로 다시 모임

- 오차 수치를 통한 위치 추정 정확도 확인

배열 인덱스	추정 오차 항목	설명
[0]	x	X축 위치의 불확실성 정도 (오차 범위)
[7]	y	Y축 위치의 불확실성 정도 (오차 범위)
[35]	yaw	회전 각도(Yaw)의 불확실성 정도

```
# 실시간 오차 분산 계수를 수치 그래프로 플롯 모니터링
ros2 run rqt_plot rqt_plot \
  /amcl_pose/pose/covariance[0] \
  /amcl_pose/pose/covariance[7] \
  /amcl_pose/pose/covariance[35]
```

Part 5. Navigation2 — 자율주행

통합 실행 런치파일을 통한 Navigation2 기동

- 패키지 통합 구성: 로봇 제조사(ROBOTIS)는 `map_server`, `amcl`, `controller/planner_server`를 한 번에 연동 기동하는 원클릭 런치 파일을 배포함

- 터미널 1 — Gazebo 가상 실습장 구동

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

- 터미널 2 — Navigation2 구동 제어부 실행

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \  
  use_sim_time:=True \  
  map:=/home/cchyun/Workspace/turtlebot3_ws/map/tb3_map.yaml
```

- 주의: `map` 인자 경로는 절대 경로를 명확하게 매핑해야 함

- 터미널 3 — 전용 프로파일 연동 RViz 시각화 도구 실행

```
rviz2 \  
-d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz
```

- Nav2에서 기본 제공하는 뷰 설정 파일을 탑재하여 전용 컨트롤 패널 플러그인이 자동으로 로드됨

- 초기 위치 설정 (2D Pose Estimate)

1. RViz 화면 상단의 `2D Pose Estimate` 버튼 클릭
2. 지도에서 로봇의 현재 위치와 방향을 마우스 드래그로 맞춤

- 목표 위치 지정 및 자율주행 실행 (Navigation2 Goal)

1. RViz 화면 상단의 `Navigation2 Goal` 버튼 클릭
2. 지도에서 로봇이 이동할 목표 위치를 클릭하고, 도착했을 때 바라볼 방향으로 드래그함
3. 명령 전달 시 내비게이션 기능이 다음 단계를 거쳐 로봇 이동 수행:
 - `planner_server`: 지도 정보를 바탕으로 목적지까지의 전체 경로(Global Plan) 계획
 - `controller_server`: 실시간 장애물 정보를 확인하며 모터 속도를 조절해 궤적(Local Plan) 추종
 - `recoveries_server`: 장애물에 막힐 경우 후진하거나 제자리 회전을 하며 탈출 시도
 - 목적지에 도착하면 자율주행 완료

- 단일 목표 지점 주행 코드 구조 (Go to Pose)

```
from nav2_simple_commander.robot_navigator import BasicNavigator
import rclpy
from geometry_msgs.msg import PoseStamped
import time

rclpy.init()
nav = BasicNavigator()
nav.waitUntilNav2Active() # Nav2 노드 상태 활성화 대기

# 목표 지점 좌표계 및 위치 지정 (시간 스탬프 주입)
goal_pose = PoseStamped()
goal_pose.header.frame_id = 'map'
goal_pose.header.stamp = nav.get_clock().now().to_msg()
goal_pose.pose.position.x = 1.5
goal_pose.pose.position.y = 0.5
goal_pose.pose.orientation.w = 1.0

nav.goToPose(goal_pose)
while not nav.isTaskComplete():
    feedback = nav.getFeedback() # 실시간 잔여 도달 거리 및 피드백 획득
    time.sleep(0.1) # CPU 과부하 방지
print("목적지에 안전하게 도달했습니다!")
rclpy.shutdown()
```

- 다중 중간 목표 경유 주행 및 목적지 시각화 코드 (Waypoints Navigation + MarkerArray)
 - 로봇 출발 전 RViz 상에 목적지 위치들을 커스텀 마커(빨간색 구형)로 띄워 시각적으로 확인하는 예제 코드

```
from nav2_simple_commander.robot_navigator import BasicNavigator
import rclpy
from geometry_msgs.msg import PoseStamped
from visualization_msgs.msg import Marker, MarkerArray
from rclpy.qos import QoSProfile, DurabilityPolicy
import time

rclpy.init()
nav = BasicNavigator()
nav.waitUntilNav2Active()

# RViz와의 마커 통신 호환성(QoS)을 맞추기 위해 TRANSIENT_LOCAL 설정 적용
qos = QoSProfile(depth=10, durability=DurabilityPolicy.TRANSIENT_LOCAL)
marker_pub = nav.create_publisher(MarkerArray, '/waypoints', qos)

# 경유할 중간 목적지 목록 설계 및 마커 생성
goals = []
marker_array = MarkerArray()
```

Python Nav2 API를 이용한 자율주행 프로그램 제어 (2/3)

```
for i, (x, y) in enumerate([(1.0, 0.0), (1.5, 1.0), (0.0, 0.0)]):
    # 주행용 목적지 좌표 설정 (시간 스탬프 주입)
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = nav.get_clock().now().to_msg()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.orientation.w = 1.0
    goals.append(pose)

    # 시각화용 커스텀 마커 설정 (빨간색 구형, 시간 스탬프 주입)
    marker = Marker()
    marker.header.frame_id = 'map'
    marker.header.stamp = nav.get_clock().now().to_msg()
    marker.ns = 'waypoints'
    marker.id = i
    marker.type = Marker.SPHERE
    marker.action = Marker.ADD
    marker.pose.position.x = x
    marker.pose.position.y = y
    marker.pose.position.z = 0.1 # 바닥에서 약간 띄움
    marker.scale.x = 0.2
    marker.scale.y = 0.2
    marker.scale.z = 0.2
    marker.color.a = 1.0 # 불투명도
    marker.color.r = 1.0 # Red
    marker_array.markers.append(marker)
```

```
# RViz에 마커 표시 (/waypoints 토픽 구독 필요)
marker_pub.publish(marker_array)

# 경유지 순차 주행 미션 실행
nav.followWaypoints(goals)
while not nav.isTaskComplete():
    time.sleep(0.1) # CPU 과부하 방지
print("모든 목적지 경로 탐색 주행을 완료했습니다!")

rclpy.shutdown()
```

Part 6. 심화 미션

- SLAM 포즈 그래프 보존 정보 쓰기

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \  
  "{filename: '/home/cchyun/tb3_cartographer_serial.pbstream'}"
```

- 저장된 pbstream 정보를 재로드하여 기동 중인 시뮬레이션 지점부터 확장 SLAM 이어서 실행

```
ros2 launch turtlebot3_cartographer cartographer.launch.py \  
  use_sim_time:=True load_state_filename:=/home/cchyun/tb3_cartographer_serial.pbstream
```

- LiDAR 스캔 정보 및 로봇 모션 변환 관계(TF) 레코딩

```
ros2 bag record /scan /tf /tf_static
```

- 기록된 백 파일 재생 및 시뮬레이션 클럭 매핑

```
ros2 bag play <녹화파일_경로> --clock
```

- 주의: `--clock` 옵션을 활용하여 파일에 기록된 당시의 모의 시간 주기를 `/clock` 토픽으로 내보내야 함.
SLAM 노드들은 `use_sim_time:=True` 옵션이 인가되어 이 재생 클럭 속도에 맞춰 동작함

- 갑작스러운 장애물 등장 시 회피 동작 확인

1. Gazebo 화면에서 물체(박스 등)를 마우스로 끌어와 로봇의 주행 경로 앞에 떨어뜨림
2. RViz 화면에서 로봇 주변의 장애물 영역(Local Costmap)에 방금 놓은 물체가 나타나는지 확인
3. 로봇이 우회 경로를 생성하여 장애물을 피하는지 확인하고, 장애물 제거 시 지도에서 지워지는지 확인

- 자동 탐색 주행 노드 활용하기

- 로봇이 스스로 장애물을 피하며 돌아다니는 노드를 실행하여 자동으로 지도 작성

- 자동 SLAM 실습 기동 연계 순서

```
# 1. Gazebo 시뮬레이션 기동
```

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

```
# 2. Cartographer SLAM 기동
```

```
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

```
# 3. 자동 회피 주행 노드 실행
```

```
ros2 run turtlebot3_gazebo turtlebot3_drive
```



- 실습 월드 파일:

- `turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_gazebo/worlds/turtlebot3_world.world`

- 구동 런치 파일:

- `turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_gazebo/launch/turtlebot3_world.launch.py`

- Nav2 매개변수 설정 파일:

- `turtlebot3_ws/src/turtlebot3/turtlebot3_navigation2/param/burger.yaml`